

AEGIS

A deny-by-default governance control plane for LLM agents on AWS

Partner brief · 10 September 2026

VERIFIED AGAINST

v0.7.0-pilot-rc1

21 / 21

full-portfolio gate, from zero, torn down
to zero residue

*One pack · one account · one region ·
synthetic data*

A reference architecture, not a product. No SLA, no support contract, no certification. Every piece of evidence in it was produced by its author.

Most “AI governance” is a prompt and a logging wrapper

Both of those live inside the thing being governed. An agent that can reason about its instructions can reason around them, and a log written after the fact records what happened rather than preventing it.



Governance in the prompt

The model is asked to comply. Compliance is a behaviour, not a control, and it degrades under adversarial input.



Governance in application code

Better — but the agent chooses which code path to call. The control is downstream of the decision it is meant to constrain.



Governance in front of the model

Authorization is evaluated by AWS before the tool runs. The agent cannot reach the decision point, so it cannot argue with it.

Aegis takes the third position. Cedar authorization, masking proven by signature, per-tenant spend ceilings, a kill switch and a WORM audit ledger sit in front of the model — so the governance cannot be talked out of by the agent it governs.

WHAT IT IS

A configurable control plane, proven end to end on one pack

13

Cedar policies at the gateway
12 forbid · 1 permit

9

CloudFormation stacks
deployed from zero

27

CDK configuration switches
network · KMS · tenancy · retention

4

vertical packs on one
hash-locked core

What it governs

- Who may call which tool, from a verified token — never the request body
- Whether the payload is provably de-identified before any consequential action
- Whether the spend ceiling still allows the call, checked before the spend
- That the record is written before the side effect, and cannot be altered after

How it is configured

- An agent manifest declares the tool targets and the guardrail
- render.py emits gateway targets, Cedar statements and guardrail config
- CDK context switches set network mode, KMS, tenancy, retention, perimeter
- A customer authors their own policies against their own manifest

The four packs share one governed-core artifact pinned by sha256. That is evidence the core ports — not evidence the other three work. Only one has ever been deployed.

One design bet, and everything else follows from it

Put the authorization decision somewhere the agent cannot reach, and make every downstream control depend on a proof rather than a claim.

1 Deny-by-default, forbid-wins

Twelve forbid policies and one permit. Any forbid that matches kills the request. A caller with a missing or malformed claim gets zero tools, not all of them — the failure mode most agent stacks get backwards.

2 Proofs, not booleans

deidentified: true from a caller is never accepted. The proof is a signed sanitized_ref, and a forged reference is refused. Raw case text and the drafted notice never cross workflow state at all.

3 A controller the model does not drive

A deterministic Step Functions machine runs a guard after every substantive step and cannot advance on unverified state. The model is a participant, never the controller.

4 The record is a control, not a log

The audit row is written before the side effect, hash-chained, append-only, with IAM Deny on update and delete — and it carries the policy version that decided the case.

Eight layers, and what each one contributes

- | | |
|--|---|
| 1 Identity & admission | Cognito issues the token carrying groups, tenant and tool entitlement. Tenancy comes from the token, never the request. |
| 2 Authorization | AgentCore Gateway (MCP, CUSTOM_JWT) + the GA Cedar engine in ENFORCE + the interceptor that sets authoritative context. |
| 3 Agent runtime & model | Runtime on the CDK-declared role, MMDSv2 required. Bedrock over PrivateLink. Guardrail ARN pinned by an IAM condition. |
| 4 Deterministic orchestration | Step Functions, ten states, a guard after every substantive step. Human sign-off with the approver re-verified. |
| 5 Governed tools | One Lambda per manifest target, exact-ARN least privilege, private subnets — measured 0 NAT, 0 IGW. |
| 6 Evidence & regulated data | Per-tenant physical isolation. Hash-chained ledger. WORM vault. Customer-managed KMS throughout. |
| 7 Capture & lineage | One account trail, log-file validation on, reconciled against audit rows in both directions. |
| 8 Operations & containment | Kill switch, per-tenant budget, Budgets ceiling. 26 named alarms in 3 severity tiers, each with a runbook entry enforced by CI. |

Full detail — every component, its purpose, what it stops, and whether it is live-verified or IaC-asserted — is in the attached AEGIS-ARCHITECTURE-VERIFIED-2026-09-10.drawio.

Twelve forbid. One permit. That asymmetry is the architecture.

12

forbid

The product. Cedar is forbid-wins: any single match kills the request, no matter how many permits fire.

1

permit

Only the door being unlocked. Grants nothing on its own — every forbid above still wins over it.

What this buys: a caller whose token is missing a claim is granted **zero** tools, not all of them. Optional context fields are presence-guarded, so an absent value fails closed — the forbid fires. The GA engine rejects an unguarded optional access outright, which is why every policy reads “context.input has X && ...”.

Proven live

13 policies reached ACTIVE, twice, in the 2026-09-10 gate — and in that gate deny-by-default reached a newly added tool with no new policy written. The decision is made by AWS, before the tool Lambda is invoked — and it is written to the hash-chained ledger with the policy version that made it (policy_version = cedar-534be1c676bb).

The thirteen policies, grouped by what they defend

A Admission

Is this caller allowed to do anything at all?

`require_tenant`
`require_entitlement`
`require_service_window`

No tenant, no entitlement, or outside the service window → zero tools. Tenancy is derived from the caller's own validated token, so cross-tenant access is impossible by construction.

B De-identification

Nothing consequential touches un-masked data

`mask_before_assess`
`mask_before_draft`
`mask_before_redetermine`
`mask_before_overpayment`

The four consequential actions are forbidden unless the payload is provably de-identified — and the proof is a signature, not a boolean.

C Separation of duties

What the agent may never do

`no_self_commit`
`no_self_fraud_referral`

Forbidden outright and hidden from the advertised tool list. The only path to a commit is the human sign-off gate.

D Purpose, spend, limits

Why, how much, how large

`consent_purpose_before_assess`
`budget_before_draft`
`amount_cap_overpayment`

Purpose limitation; a spend gate set from the live per-tenant meter, refusing before the spend; and a dollar cap above which a case needs elevated authorization.

Honest limit: these thirteen are a worked example, not a reusable library — they name benefits-specific tool actions. A customer authors their own against their own manifest, which is the real platform capability.

Masking is proven, not asserted

What most systems do

The caller passes `deidentified: true` and the platform believes it. The control is a self-report by the party it constrains.

What Aegis does

A cryptographically signed `sanitized_ref`, issued only by `mask_pii`. A forged or tampered reference is refused by the guard.

And the raw application text — and the drafted notice — never cross workflow state at all. Only opaque references do. The strict PII canary found zero leaks across Logs, X-Ray, DLQ and Step Functions history.

Why this is also the answer to “what if someone bypasses the gateway?”

We tested a control (RT-4) that restricted the runtime to gateway-only invocation. It worked — and because the gateway sits downstream of the runtime in this topology, it left the agent with no permitted invoker and failed four gate checks. It is now opt-in and off by default, so R4-2 is OPEN and we say so. The bypass is mitigated where it actually bites: the four `mask_before_*` forbids refuse the consequential actions without the signature, whoever calls. RT-4 would have closed a path; these close the action.

21 of 21, from zero, then torn down to nothing

PROVEN LIVE

**Deploy from zero**

rc=0 in 722s · nine stacks

**Guardrail on every call**

runtime_rows=20 · assessed=20

**Kill switch**

engaged, blocked, disengaged · 105s

**Unified lineage**

orphans=0 · 11 invokes / 11 rows

**Policy provenance**

policy_version in a live ledger row

**Cedar ENFORCE**

13 policies ACTIVE, twice

**Multi-tenant isolation**audit routing 12/12 · transparency 13/13
per tenant**Per-tenant budget**

refused before the spend · 373s

**Full case driven**

LIN-0E6D3D through all ten states

**Teardown**

stack, AgentCore AND connector residue

THE ONE THING THAT CHANGED SINCE 9 SEPTEMBER

A governed tool now reaches a real OAuth2 system of record. It is ours.

What is now proven

- `verify_source` is a Gateway target and holds NO client secret
- The outbound token is minted by the AgentCore Identity vault (M2M)
- The system of record verifies RS256 against the issuer's live JWKS
- Cedar denied an outsider — naming a specific policy, not refusing everyone

What it still is not

- The system of record is REAL — and it is OURS. Not a customer's.
- `connect_system_of_record` remains stubbed in the manifest, on purpose
- No customer IdP, network boundary, token lifetime or change control
- Six from-zero live runs to get this far: treat it as engineering work

Why this slide is here. Nothing in the previous edition of this deck claimed a governed connector, because until 10 September it had never once run to completion. Not one of the six failures was where it appeared to be: authentication, then the gateway authorizer, then the system of record itself — each hidden behind an error written to `/dev/null`. The gate that now proves it is stricter than the gate that used to pass it.

The two controls that stop it recurring

`CONN_deploy` used to pass on the connector's artifacts merely existing, and did so twice over a system of record that trusted nothing. It now also requires the deploy to have exited 0 and the outbound leg to have been exercised — a rule validated by replay against those two failed runs before it shipped. And CI now fails when a new silenced error path appears.

It took seven attempts. All six failures are published.

The run immediately before the pass reached the checks and failed four of them. It sits in the same commit as the pass. If you read one thing to decide whether this evidence is honest, read those two files side by side.

L60 Line-ending noise that was cosmetic — and which I first diagnosed as digest corruption and was wrong. Tested before publishing; the claim was false and the commit message was rewritten.

L64 Three consecutive wrong diagnoses of one `ConflictException`. Real cause: AgentCore policy names are unique per account and region, not per policy engine. The repo's own code comment had said so all along.

L61 The CloudTrail five-trails-per-region quota, which AWS marks not adjustable. A service limit, not a defect. Still open.

L65 / L66 RT-4 — a control that worked and made the agent unreachable. Reverted to opt-in; R4-2 reopened.

Seventy findings are recorded this way, including every one where I was wrong — and the connector on the next slide took six more from-zero runs after this one. That register is the strongest credibility asset in the repository.

Read this list before any customer conversation

NOT PROVEN

All evidence is author-produced

One AWS account. No third party has run it. No amount of further self-testing closes this.

No CUSTOMER system of record

A governed OAuth2 connector is now live and proven — but the system of record is ours. `connect_system_of_record` stays stubbed. Every live run governed synthetic data.

No account-wide prevention

No AWS Organization, therefore no SCP. A direct Bedrock call is detected, not prevented.

R4-2 is open, deliberately

Gateway-only runtime invocation was tested live and reverted. Mitigated at the gateway, not closed.

Three of four packs never deployed

Same core, propagated fixes, unit tests — zero from-zero deployments. Housing wires 3 of 17 controls.

COMPLIANCE lock proven at 1 day

The mechanism is proven; the seven-year duration is a parameter, not evidence.

No pen test, DR, SLO or ATO

None performed. The platform carries no authorization of any kind.

488 checkov findings baselined

Reviewed and gated so new findings fail CI — but the residual is real.

We said three alarms. It was twenty-two.

What we claimed

- "Three alarms. Observability is the thinnest part of the platform."
- Counted `cw.Alarm( CALL SITES` in the source - two are inside loops
- Measured the generating code instead of the deployed artifact
- Same defect as two entries already in our own register (L60, L67)

What is actually deployed

- 26 named alarms in 3 severity tiers - P1 page, P2 ticket, P3 advisory
- Latency alarmed, not merely charted. Approval backlog: 2h, was 24h
- Every alarm has a runbook entry, enforced by a test in CI
- Still open: per-tenant dashboards, canary, proof a tier fires live, DR

Why this slide is in the deck at all. A number in our own material was wrong by a factor of seven, in the direction that flattered nobody. How a team handles its own bad number is worth more signal than the number was — and the fix is not "measure more carefully next time", it is a script that reads the CloudFormation template and fails CI when these figures drift from what the brief says.

The control that stops it recurring

`tools/observability_census.py` reads the synthesized template and counts what actually deploys. `--assert-baseline` fails if the numbers drift from what the partner documents quote, so the brief and the infrastructure cannot disagree silently. Ask your SA to run it.

One thing, and it is cheap: run it yourself

“All of the evidence is ours, which means none of it counts. We want your SA to run the verification protocol in a clean account. It is scripted, it takes about half a day, and it either reproduces our results or it does not.”

1	Clone the tag	10 min	v0.7.0-pilot-rc1. Confirm tag == main and the RELEASE file inside the tag names the tag.
2	Run the offline suite	20 min	510 pass, 4 skip (514 collected). Confirm the CI security gates are blocking, not report-only.
3	Read a pass and a failure	20 min	A pass and the failure before it, side by side, in the same commit. Cheapest high-signal step in the list.
4	Deploy from zero	~2 hr	Into a clean account you control, following docs/INDEPENDENT-VERIFICATION.md.
5	Run the gate yourself	~1.5 hr	Record the result — including if it fails. That is the deliverable either way.
6	Try to break it	~1 hr	Forge a sanitized_ref. Call a tool with no entitlement. Call finalize_determination directly.
7	Tear down	30 min	Confirm zero stack and zero AgentCore residue in your account.

One prerequisite we owe you: the account needs CloudTrail headroom. The limit is five trails per region and AWS marks it not adjustable. Each deployed pack needs two.

Verify, then extend, then a customer — in that order



Verify

Weeks 1–2

Partner SA reproduces the gate in a clean account, including the break-it step.

Gate: it reproduces, or we learn why not.



Extend

Weeks 3–8

The same connector proof against a real EXTERNAL system of record — ours is proven, a customer's is not. Live re-gate of a second pack. Observability uplift. Organization + SCP if you can supply an OU.

Gate: a system of record we do not own is governed.



Pilot

Quarter 2

Named customer. Supervised pilot on de-identified data. Customer's own security engineering, testing and authorization.

Gate: the customer's AO accepts the compensating-control statement.



Production

Later

Not in scope for this conversation. Pen test, DR, SLO, IdP round-trip, ATO, formal authorization.

—

We are deliberately NOT asking you to find a customer yet. That would be asking you to put your name on synthetic, author-produced evidence.

Where this actually stands

What it is

A credible governance control plane at reference-architecture maturity, with an unusually well-documented gap list — and further along than most teams will have built themselves.

Good for: architecture workshops, a CISO/AO conversation, a supervised pilot on synthetic or de-identified data.

What it is not

Not a product. Not an AWS service. Not certified, authorized or ATO'd. Not ready for real regulated data. Not independently verified by anyone.

Say this first. It costs nothing, and it buys you the rest of the conversation.

The one thing we need from you

Half a day of an SA's time, in an account we do not control, to find out whether any of this survives contact with someone who is not us.

AEGIS

A deny-by-default governance control plane for LLM agents on AWS

Partner brief · 10 September 2026

VERIFIED AGAINST

v0.7.0-pilot-rc1

21 / 21

full-portfolio gate, from zero, torn down
to zero residue

One pack · one account · one region ·
synthetic data

A reference architecture, not a product. No SLA, no support contract, no certification. Every piece of evidence in it was produced by its author.

THE MOVE

Establish that you will qualify your own claims before anyone has to ask you to.

SAY

- Open with the scope, not the number: "Before I show you anything - one pack, one AWS account, one region, synthetic data, and every piece of evidence in here was produced by me, on my own account."
- Then the number: 21 of 21, from an empty account back to an empty account, 10 September.
- Then the frame, verbatim: "This is a reference architecture, not a product. No SLA, no support contract, no certification."
- Tell them the ask in the first two minutes so the rest of the meeting has a shape: "I am going to ask you for half a day of an SA's time. That is the whole ask."

DON'T

- Don't lead with 21/21. The number is only credible because the qualification arrives in the same breath.
- Don't say "platform" as though it is a product, and don't let "verified" stand alone - verified BY ME is the accurate phrase.

Q&A

Q: What does "from zero" actually mean?

A: The account is empty when the run starts and empty when it ends, and both facts are established by asking AWS directly - not by trusting the teardown script's own verdict. That distinction is not pedantry: a teardown once reported CLEAN over a live credential provider holding a client secret. That is why there are now four separate residue checks.

Q: Is 21 of 21 better than the 17 of 17 I saw last week?

A: Same platform, four more checks. The four are a governed system-of-record connector and its residue sweep. The total going up is not the story - what the new checks measure is. I will spend a slide on exactly that.

Q: Who is using this today?

A: Nobody. That is the honest answer and it is the reason for this meeting.

Q: Why should I spend time on a reference architecture?

A: Because what your customers are shipping right now is a system prompt and a log file. This is the same problem solved somewhere the agent cannot argue with it. And the ask is half a day, not a commitment.

THE PROBLEM

Most “AI governance” is a prompt and a logging wrapper

Both of those live inside the thing being governed. An agent that can reason about its instructions can reason around them, and a log written after the fact records what happened rather than preventing it.



Governance in the prompt

The model is asked to comply. Compliance is a behaviour, not a control, and it degrades under adversarial input.



Governance in application code

Better — but the agent chooses which code path to call. The control is downstream of the decision it is meant to constrain.



Governance in front of the model

Authorization is evaluated by AWS before the tool runs. The agent cannot reach the decision point, so it cannot argue with it.

Aegis takes the third position. Cedar authorization, masking proven by signature, per-tenant spend ceilings, a kill switch and a WORM audit ledger sit in front of the model — so the governance cannot be talked out of by the agent it governs.

THE MOVE

Make the category distinction land - governance **INSIDE** the thing being governed is not governance - so that everything after this slide has somewhere to sit.

SAY

- Both of the common approaches live inside the thing they are supposed to constrain.
- A prompt **ASKS** the model to comply. Compliance is a behaviour, not a control, and behaviours degrade under adversarial input.
- Application-code controls are genuinely better - but the agent chooses which code path to call, so the control sits downstream of the decision it is meant to constrain.
- Land the last line on the slide. If they remember one sentence from the whole deck, it is that the governance cannot be talked out of by the agent it governs.

DON'T

- Don't caricature the alternatives. Plenty of good teams are at position two, and saying so buys you credibility for the rest.

Q&A

Q: Don't Bedrock Guardrails already do this?

A: Guardrails filter content, and we use them - every runtime model call in the gate was guardrail-assessed, 20 of 20. What a guardrail does not decide is whether **THIS** human may invoke **THIS** tool against **THIS** tenant's data for **THIS** purpose within **THIS** budget. That is authorization, and it is a different question. Guardrails are one control inside the perimeter, not the perimeter.

Q: Isn't this just an API gateway with authentication?

A: The difference is where the decision is made and what it is made from. Cedar evaluates the real human's token at the AgentCore Gateway before any tool Lambda is invoked, using context the caller cannot supply - tenant, budget and service window are injected server-side and any caller-supplied value is stripped first.

Q: What if the model itself is the thing I can't trust?

A: Then you should like this design. The model never holds a credential, never chooses the control path, and cannot commit anything. Every consequential action terminates at a human gate where a different verified identity approves.

Q: Where does this break down?

A: When someone reaches the compute without going through the gateway. I have a slide on that and the answer is a reopened finding, not a fix.

WHAT IT IS

A configurable control plane, proven end to end on one pack

13

Cedar policies at the gateway
12 forbid · 1 permit

9

CloudFormation stacks
deployed from zero

27

CDK configuration switches
network · KMS · tenancy · retention

4

vertical packs on one
hash-locked core

What it governs

- Who may call which tool, from a verified token — never the request body
- Whether the payload is provably de-identified before any consequential action
- Whether the spend ceiling still allows the call, checked before the spend
- That the record is written before the side effect, and cannot be altered after

How it is configured

- An agent manifest declares the tool targets and the guardrail
- render.py emits gateway targets, Cedar statements and guardrail config
- CDK context switches set network mode, KMS, tenancy, retention, perimeter
- A customer authors their own policies against their own manifest

The four packs share one governed-core artifact pinned by sha256. That is evidence the core ports — not evidence the other three work. Only one has ever been deployed.

THE MOVE

Give them the scale of the thing - and disqualify the fourth number yourself, out loud, before anyone does it for you.

SAY

- Four numbers. Three are straightforward: 13 Cedar policies at the gateway, 9 CloudFormation stacks deployed from zero, 27 CDK switches that shape a deployment without editing code.
- The fourth needs the caveat in the same breath: four packs on one hash-locked core is evidence the core PORTS. It is not evidence the other three work. Only one has ever been deployed.
- The real platform capability is the configuration path: an agent manifest declares the tool targets, render.py emits the gateway targets, the Cedar statements and the guardrail config. That is what a customer would use.

DON'T

- Don't let "four verticals" sit in the room unqualified for even a few seconds. It is the single easiest thing in this deck to over-read.

Q&A

Q: Can I stand this up for a customer in a different domain?

A: The scaffold, yes - that is what the manifest and the render pipeline are for. The thirteen policies, no: they name benefits-specific tool actions like assess-eligibility. A customer authors their own. "Configurable governance platform" is true; "thirteen reusable policies" is not, and I try not to blur them.

Q: What is a CDK context switch - is that an AWS feature?

A: No, it is our infrastructure code. The 27 switches cover network mode, KMS mode, retention profile, identity mode, tenancy, perimeter, account-wide capture, model logging and the spend ceiling.

Q: If only one pack is proven, why ship four?

A: Because the property being demonstrated is that the core is consumed as a pinned, hash-verified dependency rather than copied. That matters: we found the same control present in two packs and missing from two others while all four integrity locks recorded the same hash and every CI run was green. The dependency prevents that. The other three get every fix with unit tests and a cross-repo parity check - but no live gate.

Q: Pinned by sha256 - what happens when you release a new core?

A: The consumer bumps the version and the hash together, and pip refuses the install if the artifact at that URL ever changes. That is the property that makes it a dependency rather than a copy with extra steps.

One design bet, and everything else follows from it

Put the authorization decision somewhere the agent cannot reach, and make every downstream control depend on a proof rather than a claim.

- | | | |
|----------|--|---|
| 1 | Deny-by-default, forbid-wins | Twelve forbid policies and one permit. Any forbid that matches kills the request. A caller with a missing or malformed claim gets zero tools, not all of them — the failure mode most agent stacks get backwards. |
| 2 | Proofs, not booleans | deidentified: true from a caller is never accepted. The proof is a signed sanitized_ref, and a forged reference is refused. Raw case text and the drafted notice never cross workflow state at all. |
| 3 | A controller the model does not drive | A deterministic Step Functions machine runs a guard after every substantive step and cannot advance on unverified state. The model is a participant, never the controller. |
| 4 | The record is a control, not a log | The audit row is written before the side effect, hash-chained, append-only, with IAM Deny on update and delete — and it carries the policy version that decided the case. |

THE MOVE

Show that the architecture follows from one decision, not from a shopping list of controls.

SAY

- The bet: put the authorization decision somewhere the agent cannot reach, and make every downstream control depend on a PROOF rather than a CLAIM.
- Bet 2 is the one worth defending in detail - it is what makes reverting RT-4 defensible when we get to that slide, so plant it now.
- Land bet 4 deliberately: the audit row is a control, not a log. It is written BEFORE the side effect, and a consequential commit requires the WORM copy to be durable - not just the ledger write. That was a fault-semantics finding from an external review, not something we got right first time.

DON'T

- Don't rush bet 3. "The model is a participant, never the controller" is the sentence that separates this from an agent framework with logging.

Q&A

- Q: Deny-by-default sounds expensive to operate. How does anyone get work done?
- A: One permit grants the caseworker role the governed tools; the twelve forbids constrain how they may be used. The friction lands on missing claims and missing proofs, which is exactly where you want it. In the gate run a legitimate reviewer drove a full case through all ten workflow states without touching a policy.
- Q: What happens when a Cedar policy is simply wrong?
- A: Two answers. First, the decision is written to the ledger with the policy version that made it, so "which policy decided this case?" is answerable from the record rather than from someone's memory. Second, the policies are ours and a customer replaces them - that is the design, not a dodge.
- Q: Writing the record before the side effect means you record things that never happened.
- A: Yes, deliberately. An INTENT row with no matching outcome is a detectable anomaly and the lineage proof looks for exactly that. The alternative - writing after - means a crash between the action and the log leaves no record of the action at all, which is the failure mode an auditor actually cares about.
- Q: "A forged reference is refused" - refused by what?
- A: By the workflow guard, before the consequential step runs. The signature is issued only by the masking control. I will spend a whole slide on this because it is the strongest thing in the system.

COMPONENTS

Eight layers, and what each one contributes

1	Identity & admission	Cognito issues the token carrying groups, tenant and tool entitlement. Tenancy comes from the token, never the request.
2	Authorization	AgentCore Gateway (MCP, CUSTOM_JWT) + the GA Cedar engine in ENFORCE + the interceptor that sets authoritative context.
3	Agent runtime & model	Runtime on the CDK-declared role, MMDSv2 required. Bedrock over PrivateLink. Guardrail ARN pinned by an IAM condition.
4	Deterministic orchestration	Step Functions, ten states, a guard after every substantive step. Human sign-off with the approver re-verified.
5	Governed tools	One Lambda per manifest target, exact-ARN least privilege, private subnets — measured 0 NAT, 0 IGW.
6	Evidence & regulated data	Per-tenant physical isolation. Hash-chained ledger. WORM vault. Customer-managed KMS throughout.
7	Capture & lineage	One account trail, log-file validation on, reconciled against audit rows in both directions.
8	Operations & containment	Kill switch, per-tenant budget, Budgets ceiling. 26 named alarms in 3 severity tiers, each with a runbook entry enforced by CI.

Full detail — every component, its purpose, what it stops, and whether it is live-verified or IaC-asserted — is in the attached AEGIS-ARCHITECTURE-VERIFIED-2026-09-10.drawio.

THE MOVE

Orient them without reading eight bullets. Then hand them the artifact that proves you are not hiding the weak parts.

SAY

- Do not read all eight. Pick three for this audience: layer 2 (authorization), layer 6 (evidence), layer 8 (containment). Offer the rest on request.
- Layer 5 has a number worth saying out loud: zero NAT gateways, zero internet gateways - MEASURED from the synthesized template, not asserted in prose.
- Point at the attached architecture file and say the thing that makes it credible: every component carries a status - LIVE, IAC-asserted, or OFF - and the OFF and adopter-owned boxes are drawn the same size and in the same places as the live ones. A diagram that showed only what works would be a sales diagram.

DON'T

- Don't claim "Bedrock in your VPC". Say PrivateLink and mean PrivateLink.

Q&A

Q: Bedrock over PrivateLink - so the model runs inside my VPC?

A: No, and the distinction matters enough that I correct it whenever I hear it. PrivateLink means traffic to Bedrock does not traverse the public internet. It does not mean Bedrock runs in your VPC.

Q: What is the difference between IaC-asserted and live-verified?

A: Live-verified means a from-zero run exercised it and a named gate check proves it. IaC-asserted means CDK builds it and a synthesis test asserts it, but no live run has demonstrated it. WAF on the auth front door is an example of the second, and it is drawn that way.

Q: 26 alarms - are they wired to anything, or is that a dashboard number?

A: Three severity tiers, each with its own SNS topic: P1 page, P2 ticket, P3 advisory. Every alarm has an entry in the incident runbook, enforced by a CI test - a new alarm without a documented first action fails the build. What I cannot tell you is that any tier has ever fired in a real incident. That is on the not-proven list and I will show you.

Q: Per-tenant isolation - is that a partition key?

A: No. Each tenant gets its own tables and its own WORM bucket, not a shared table with a tenant column. And the tenant is DERIVED from the verified token by the gateway interceptor, never read from the request body, so cross-tenant access is impossible by construction rather than by check.

Twelve forbid. One permit. That asymmetry is the architecture.

12

forbid

The product. Cedar is forbid-wins: any single match kills the request, no matter how many permits fire.

1

permit

Only the door being unlocked. Grants nothing on its own — every forbid above still wins over it.

What this buys: a caller whose token is missing a claim is granted **zero** tools, not all of them. Optional context fields are presence-guarded, so an absent value fails closed — the forbid fires. The GA engine rejects an unguarded optional access outright, which is why every policy reads "context.input has X && ...".

Proven live

13 policies reached ACTIVE, twice, in the 2026-09-10 gate — and in that gate deny-by-default reached a newly added tool with no new policy written. The decision is made by AWS, before the tool Lambda is invoked — and it is written to the hash-chained ledger with the policy version that made it (policy_version = cedar-534be1c676bb).

THE MOVE

Make them feel why the asymmetry is load-bearing, not stylistic.

SAY

- Cedar is forbid-wins: any single forbid that matches kills the request no matter how many permits fire. So the twelve forbids are the product and the one permit is only the door being unlocked.
- The consequence is the failure mode most agent stacks get backwards: a caller with a missing or malformed claim is granted ZERO tools, not all of them.
- New in this run and worth saying: deny-by-default reached a NEWLY ADDED tool with no new policy written. That is the portability claim doing real work rather than being asserted.

DON'T

- Don't say "deny by default" and move on. The interesting property is that it is selective - see the first question below, and get there before they do.

Q&A

Q: What if a policy has a bug and just denies everything? How would you know?

A: That happened, and it is the most useful thing on this slide. On one run the outsider AND the legitimate reviewer were denied by the same string. "Outsider denied" proved nothing, because everything was denied - and widening the check to make it pass would have turned it green while proving less than nothing. A deny-by-default claim is only evidence when the denial is SELECTIVE, so the proof now requires the denial to name a specific policy.

Q: Why does every policy read "context.input has X && ..."?

A: Because the GA Cedar engine rejects an unguarded optional-attribute access outright. The useful consequence is that an absent value fails CLOSED - the forbid fires.

Q: Can a caller influence the context Cedar evaluates?

A: No. The interceptor strips caller-supplied consent, purpose, budget_ok and service-window values and injects only server-derived ones - the real clock, the live meter, a pack resolver. That came out of an external review round; the earlier version trusted caller assertions for some of those fields.

Q: Where is the decision actually made?

A: In AWS's Cedar engine at the AgentCore Gateway, before the tool Lambda is invoked. Not in our code, and not anywhere the agent can reach.

The thirteen policies, grouped by what they defend

A

Admission

Is this caller allowed to do anything at all?

`require_tenant`
`require_entitlement`
`require_service_window`

No tenant, no entitlement, or outside the service window → zero tools. Tenancy is derived from the caller's own validated token, so cross-tenant access is impossible by construction.

B

De-identification

Nothing consequential touches un-masked data

`mask_before_assess`
`mask_before_draft`
`mask_before_redetermine`
`mask_before_overpayment`

The four consequential actions are forbidden unless the payload is provably de-identified — and the proof is a signature, not a boolean.

C

Separation of duties

What the agent may never do

`no_self_commit`
`no_self_fraud_referral`

Forbidden outright and hidden from the advertised tool list. The only path to a commit is the human sign-off gate.

D

Purpose, spend, limits

Why, how much, how large

`consent_purpose_before_assess`
`budget_before_draft`
`amount_cap_overpayment`

Purpose limitation; a spend gate set from the live per-tenant meter, refusing before the spend; and a dollar cap above which a case needs elevated authorization.

Honest limit: these thirteen are a worked example, not a reusable library — they name benefits-specific tool actions. A customer authors their own against their own manifest, which is the real platform capability.

THE MOVE

Show breadth in four groups, then disqualify the reusability claim yourself.

SAY

- Walk the four groups in a sentence each. Do not read thirteen policy names.
- Group B is the one to dwell on, because it sets up the next slide.
- Group C is the separation-of-duties answer: the agent can never commit a determination or raise a fraud referral - forbidden AND hidden from the advertised tool list.
- Finish on the honest limit, out loud: these thirteen are a worked example, not a reusable library. A customer authors their own against their own manifest, and THAT is the platform capability.

DON'T

- Don't let "thirteen policies" be heard as "thirteen policies you get for free".

Q&A

Q: The overpayment cap is \$5,000 - where did that come from?

A: It is illustrative. The program rules and thresholds in this pack are illustrative federal defaults. They need a domain owner who can say what the authoritative values are and sign off on them, and that is pilot work, not done work.

Q: Why hide `finalize_determination` rather than just forbid it?

A: Both, deliberately. It is forbidden by Cedar AND absent from the advertised tool list, so the agent cannot select it in the first place. The forbid is the control; the hiding removes the temptation. Every new high-risk action gets the same treatment.

Q: Is a service window really a security control?

A: It is a scope control, and the part that makes it meaningful is that it is authoritative: set by the interceptor from the real request time, never from the caller. It is scoped to the four consequential actions because the built-in AgentCore actions carry no input in their context.

Q: How does a customer write their own policies - do they need Cedar expertise?

A: They declare the policies in the agent manifest and the render pipeline emits the Cedar statements with the gateway ARN substituted. Cedar expertise helps, but the manifest is the interface. I would not pretend it is no-code.

THE STRONGEST CONTROL IN THE SYSTEM

Masking is proven, not asserted

What most systems do

The caller passes deidentified: true and the platform believes it. The control is a self-report by the party it constrains.

What Aegis does

A cryptographically signed sanitized_ref, issued only by mask_pii. A forged or tampered reference is refused by the guard.

And the raw application text — and the drafted notice — never cross workflow state at all. Only opaque references do. The strict PII canary found zero leaks across Logs, X-Ray, DLQ and Step Functions history.

Why this is also the answer to “what if someone bypasses the gateway?”

We tested a control (RT-4) that restricted the runtime to gateway-only invocation. It worked — and because the gateway sits downstream of the runtime in this topology, it left the agent with no permitted invoker and failed four gate checks. It is now opt-in and off by default, so R4-2 is OPEN and we say so. The bypass is mitigated where it actually bites: the four mask_before_* forbids refuse the consequential actions without the signature, whoever calls. RT-4 would have closed a path; these close the action.

THE MOVE

Land the strongest control - and then hand them the bypass question before they think to ask it. Answering with a reopened finding rather than a fix is the credibility move of the deck.

SAY

- Most systems accept deidentified: true from the caller - a self-report by the party the control constrains. Ours is a signed sanitized_ref, issued only by mask_pii, and a forged or tampered reference is refused by the guard.
- The part not on the slide: raw application text and the drafted notice never cross workflow state AT ALL. Only opaque references do. Pass-by-reference, not pass-by-value.
- Strict PII canary: zero leaks across CloudWatch Logs, X-Ray, the DLQ and Step Functions execution history.
- Then volunteer the bypass question yourself: "The obvious attack is, what if someone skips the gateway?" Give the RT-4 answer below.

DON'T

- Don't present RT-4 as a fix. It is an OPEN finding, R4-2, and saying so is worth more than claiming it closed.

Q&A

Q: What if someone bypasses the gateway and invokes the runtime directly?

A: We built exactly that control - RT-4, gateway-only runtime invocation. It worked. And because the gateway sits downstream of the runtime in this topology, it left the agent with no permitted invoker and failed four gate checks. So it is now opt-in and off by default, and R4-2 is OPEN in the register. The mitigation is these four mask_before_* forbids: they refuse the consequential ACTIONS unless the payload is provably de-identified, whoever calls. RT-4 would have closed a path; these close the action - which is the stronger place to enforce, and it is live-proven rather than unit-tested.

Q: What signs the sanitized_ref, and what happens if that secret leaks?

A: HMAC-SHA256 with a per-deployment shared secret in AWS Secrets Manager. Symmetric is deliberate: signer and verifier are two Lambdas in the SAME deployment and account, already inside one trust boundary. Where a boundary is actually crossed - the connector reaching the system of record - it is asymmetric RS256 against published JWKS. If the secret leaked you could forge a reference, which is why it is in Secrets Manager rather than the template, cached only on a SUCCESSFUL read, and fails closed when unreadable. A plaintext env fallback exists for disposable sandbox validation only. Attack it - it is on the break-it list.

Q: Why not KMS asymmetric signing everywhere?

A: It would be machinery without a threat to answer: inside one account the two parties already share a trust boundary, so asymmetric buys non-repudiation between parties that do not need it. The moment the boundary is real, the design switches. If your threat model says otherwise I would genuinely like to hear it.

Q: Comprehend is not perfect at PII detection.

A: Agreed, and the platform does not rely on it alone. There is a Luhn-checked regex backstop for SSN, email, phone, IP and card, and UTF-8 byte-window chunking so a record larger than Comprehend's synchronous size limit does not get its tail returned unmasked. That was a real defect, found and fixed in core 1.10.0.

Q: Zero leaks across four surfaces - how hard did you look?

A: A strict canary with known markers, checked in Logs, X-Ray, the DLQ and Step Functions history. It is a canary, not a pen test. There has been no penetration test at all and that is on the not-proven list.

WHAT IS PROVEN

21 of 21, from zero, then torn down to nothing

PROVEN LIVE

✓ Deploy from zero	rc=0 in 722s · nine stacks	✓ Cedar ENFORCE	13 policies ACTIVE, twice
✓ Guardrail on every call	runtime_rows=20 · assessed=20	✓ Multi-tenant isolation	audit routing 12/12 · transparency 13/13 per tenant
✓ Kill switch	engaged, blocked, disengaged · 105s	✓ Per-tenant budget	refused before the spend · 373s
✓ Unified lineage	orphans=0 · 11 invokes / 11 rows	✓ Full case driven	LIN-0E6D3D through all ten states
✓ Policy provenance	policy_version in a live ledger row	✓ Teardown	stack, AgentCore AND connector residue

Scope of that claim, exactly: one pack, one AWS account, one region, two tenants, synthetic data. Run ben-fpg, 10 September 2026. Evidence: evidence/FULL-PORTFOLIO-GATE-benefits_runtime_agent.json

THE MOVE

Give them the result with its scope attached, and make it easy to check rather than easy to believe.

SAY

- Read the scope line FIRST, then the grid. One pack, one account, one region, two tenants, synthetic data, run ben-fpg.
- Do not read ten tiles. Pick three this audience cares about and say the CHECK NAME out loud
 - they can grep for it in the evidence file.
- Offer the evidence file by name and offer the failed run beside it. That offer is more persuasive than the grid.

DON'T

- Don't let "proven" drift into "certified". Nothing here is certified.

Q&A

Q: Can I see the evidence file?

A: Yes - it is committed in the repository, and so is the failed run immediately before it, in the same commit. I would rather you read those two side by side than take this grid at face value. It is the cheapest high-signal thing you can do.

Q: What does "audit routing 12/12, transparency 13/13 per tenant" actually mean?

A: Two tenants with physically separate stores. Twelve isolation assertions confirm each tenant's records land in that tenant's own ledger, WORM vault and approvals table - and that the other tenant cannot reach them. Thirteen transparency assertions confirm the correlation set - tenant, session, trace, MCP session, execution, request, case - is present and joinable, per tenant.

Q: Nine stacks in 722 seconds - cold account or warm?

A: Cold. Empty account with CDK bootstrapped, no prior resources. The preflight check refuses to deploy at all if any gateway, policy engine or agent runtime carrying the prefix already exists - that check exists because a leftover policy engine from a sibling pack once held the policy names and produced three wrong diagnoses.

Q: What is NOT in this grid?

A: Real data, a customer system of record, an Organization-level perimeter, and any verification by someone who is not me. The next-but-one slide is that list, and it is longer than this one.

THE ONE THING THAT CHANGED SINCE 9 SEPTEMBER

A governed tool now reaches a real OAuth2 system of record. It is ours.

What is now proven

- `verify_source` is a Gateway target and holds NO client secret
- The outbound token is minted by the AgentCore Identity vault (M2M)
- The system of record verifies RS256 against the issuer's live JWKS
- Cedar denied an outsider — naming a specific policy, not refusing everyone

What it still is not

- The system of record is REAL — and it is OURS. Not a customer's.
- `connect_system_of_record` remains stubbed in the manifest, on purpose
- No customer IdP, network boundary, token lifetime or change control
- Six from-zero live runs to get this far: treat it as engineering work

Why this slide is here. Nothing in the previous edition of this deck claimed a governed connector, because until 10 September it had never once run to completion. Not one of the six failures was where it appeared to be: authentication, then the gateway authorizer, then the system of record itself — each hidden behind an error written to `/dev/null`. The gate that now proves it is stricter than the gate that used to pass it.

The two controls that stop it recurring

`CONN_deploy` used to pass on the connector's artifacts merely existing, and did so twice over a system of record that trusted nothing. It now also requires the deploy to have exited 0 and the outbound leg to have been exercised — a rule validated by replay against those two failed runs before it shipped. And CI now fails when a new silenced error path appears.

THE MOVE

This is the only substantive change since the last edition of this deck. Claim it precisely - and say "it is ours" before they work it out for themselves.

SAY

- Left column, briskly: `verify_source` is a Gateway target; it holds NO client secret; the outbound OAuth2 token is minted by the AgentCore Identity vault; the system of record verifies RS256 against the issuer's live JWKS; an outsider calling it was denied by Cedar, naming a specific policy rather than refusing everyone alike.
- Then, before anyone asks, the right column: the system of record is REAL - and it is OURS. `connect_system_of_record`, the one that would reach a state benefits system, stays stubbed.
- The six-runs story is the credibility, not the pass. Not one of the six failures was where it appeared to be.
- Land the standing rule in the bottom band: existence is not function. An artifact check that never exercises the thing is not evidence that the thing works.

DON'T

- Don't say "we now have a system-of-record connector" without the second half of the sentence in the same breath. That is the single most over-readable claim in the deck.

Q&A

Q: If it is your own mock, what has it actually proven?

A: That the governed path works end to end against a real OAuth2 protocol: a token from a vault rather than a secret in code, RS256 verified against live published keys, a 401 for an unauthenticated caller, and Cedar denying an outsider selectively. What it has NOT proven is anything about a customer's schema, entitlements, latency or change control. That is the next milestone and it needs a counterparty - which is part of what I am asking you for.

Q: Six runs. Should I be worried about the engineering?

A: You should judge it on the diagnosis, and I would say that either way. The failures were: proof users that a code path never creates, an authorizer that allow-lists exactly one client, and a Lambda environment silently left empty because a label contained a comma. None of those is visible in a design review. Three of the six were hidden behind an error written to `/dev/null` - and that defect class now fails CI.

Q: Your own gate passed twice over a broken connector. Why would I trust the gate?

A: You should not trust it further than its rule allows, which is why the rule changed. It used to pass on the connector's artifacts merely EXISTING. It now also requires the deploy to have exited 0 and the outbound leg to have been exercised - and that rule was replayed against the committed evidence of the two broken runs BEFORE it shipped. It fails both and passes the proven one, where the old rule passes all three. A rule that cannot fail a run you know was broken is not a control.

Q: How long to point this at a real system of record?

A: Honestly: the engineering is days, the integration is weeks, and the variable is the counterparty - their IdP, their network boundary, their token lifetimes, their change control. I will not quote you a date that depends on someone else's security review.

Q: Why is `connect_system_of_record` still stubbed if the connector works?

A: Because they are different claims. One is "the governed path can reach an OAuth2 API" and it is proven. The other is "a customer's system of record has been governed" and it is not. The manifest records the second as stubbed so nobody can read the first as the second.

WHY YOU SHOULD BELIEVE THE NUMBER ABOVE

It took seven attempts. All six failures are published.

The run immediately before the pass reached the checks and failed four of them. It sits in the same commit as the pass. If you read one thing to decide whether this evidence is honest, read those two files side by side.

L69

Line-ending noise that was cosmetic — and which I first diagnosed as digest corruption and was wrong. Tested before publishing; the claim was false and the commit message was rewritten.

L64

Three consecutive wrong diagnoses of one ConflictException. Real cause: AgentCore policy names are unique per account and region, not per policy engine. The repo's own code comment had said so all along.

L61

The CloudTrail five-trails-per-region quota, which AWS marks not adjustable. A service limit, not a defect. Still open.

L65 / L66

RT-4 — a control that worked and made the agent unreachable. Reverted to opt-in; R4-2 reopened.

Seventy findings are recorded this way, including every one where I was wrong — and the connector on the next slide took six more from-zero runs after this one. That register is the strongest credibility asset in the repository.

THE MOVE

Convert the failure list from an apology into the reason the pass is believable.

SAY

- The point of this slide is not the four findings. It is that the run that FAILED sits in the same commit as the run that passed.
- Say it plainly: if you read one thing to decide whether this evidence is honest, read those two files side by side.
- Seventy findings are recorded this way, including every one where I was wrong - and the connector on the previous slide took six more runs after this one.

DON'T

- Don't perform humility. State the findings flatly and let them do the work.

Q&A

Q: Isn't publishing your own failures a bit performative?

A: Fair challenge, and the test is whether the register changes behaviour. It does: several entries produced CI controls that now fail builds - the account-id gate, the doc-count gate, the silenced-error gate, the stale-checkout gate. If it were performative there would be no code attached to it.

Q: L61, the CloudTrail quota - is that going to bite us?

A: Yes, and it is the one prerequisite I owe you before you start. Five trails per region, AWS marks it not adjustable, and each deployed pack needs two. Check your account's headroom first. The support case needs a human in the console and a Business or Enterprise support plan, which is why it is still open.

Q: You diagnosed one error wrongly three times in a row. Why does that build confidence?

A: Because the entry says so, names the real cause - AgentCore policy names are unique per account and region, not per policy engine - and records that the repo's own code comment had said it the whole time. The alternative version of this project is the one where that entry does not exist and you find out later.

Q: How do I know the register is complete?

A: You don't, and I can't prove a negative. What I can show you is that entries get added when they are embarrassing, that the failed runs are committed rather than described, and that several entries are of the form "a control I wrote was wrong".

WHAT IS NOT PROVEN

Read this list before any customer conversation

NOT PROVEN

All evidence is author-produced

One AWS account. No third party has run it. No amount of further self-testing closes this.

No CUSTOMER system of record

A governed OAuth2 connector is now live and proven — but the system of record is ours. connect_system_of_record stays stubbed. Every live run governed synthetic data.

No account-wide prevention

No AWS Organization, therefore no SCP. A direct Bedrock call is detected, not prevented.

R4-2 is open, deliberately

Gateway-only runtime invocation was tested live and reverted. Mitigated at the gateway, not closed.

Three of four packs never deployed

Same core, propagated fixes, unit tests — zero from-zero deployments. Housing wires 3 of 17 controls.

COMPLIANCE lock proven at 1 day

The mechanism is proven; the seven-year duration is a parameter, not evidence.

No pen test, DR, SLO or ATO

None performed. The platform carries no authorization of any kind.

488 checkov findings baselined

Reviewed and gated so new findings fail CI — but the residual is real.

This list is longer than the proven list, and it is in the deck on purpose. NOT-CLAIMS.md governs every other page in the repository.

THE MOVE

Slow down. This list is longer than the proven list on purpose, and the first tile is the whole reason for the meeting.

SAY

- Take this slowly and do not skip tiles to save time. Skipping is what gets found later.
- Tile one is the meeting: all evidence is author-produced, one AWS account, no third party has run it, and no amount of further self-testing closes it.
- Say the closing line as a commitment: NOT-CLAIMS.md governs every other page in the repository - if anything I have said today reads stronger than that page, that page wins.

DON'T

- Don't soften any tile with "but". Each one can stand on its own.

Q&A

Q: 488 checkov findings is a lot.

A: It is, and the number is real. They concentrate in Lambda concurrency limits, DLQs, VPC attachment, log-group and environment KMS, DynamoDB CMK and PITR. The baseline is reviewed and gated so NEW findings fail CI - the residual does not grow silently. Burning it down to a justified residual is an open P1.

Q: No Organization means no SCP. How big a problem is that in public sector?

A: Frequently the question that decides the deal. Today a non-governed principal calling Bedrock directly is DETECTED, not prevented - there is a metric filter and a bypass alarm. The SCP and the VPC-endpoint policy ship in the repo as a lint-validated template that has never been tested in a real OU. If you can supply an OU, that is the cheapest unblock on my entire list.

Q: Why prove the COMPLIANCE lock at one day rather than seven years?

A: Because at seven years I could not tear it down, and I would be showing you an untestable claim. The mechanism IS proven: BypassGovernanceRetention is refused and the deployment's own teardown fails. The duration is a parameter. Stating it any other way invites an auditor to find the gap themselves.

Q: Housing wires 3 of 17 controls - why is it in the portfolio?

A: It should not be presented as a peer of the other three, and this deck does not. It is at the July Gate-B level and it is tracked as PAR-2 on the backlog.

Q: R4-2 open "deliberately" sounds like a rationalisation.

A: Judge it on the mitigation. The consequential actions are refused without a cryptographic proof of de-identification, whoever calls them. That is a stronger place to enforce than the invocation path, and it is live-proven. If your SA can get a consequential action through without that proof, that is the finding of the engagement and I want it.

A NUMBER WE GOT WRONG, AND WHAT WE DID ABOUT IT

We said three alarms. It was twenty-two.

What we claimed

- "Three alarms. Observability is the thinnest part of the platform."
- Counted `cw.Alarm()` CALL SITES in the source - two are inside loops
- Measured the generating code instead of the deployed artifact
- Same defect as two entries already in our own register (L60, L67)

What is actually deployed

- 26 named alarms in 3 severity tiers - P1 page, P2 ticket, P3 advisory
- Latency alarmed, not merely charted. Approval backlog: 2h, was 24h
- Every alarm has a runbook entry, enforced by a test in CI
- Still open: per-tenant dashboards, canary, proof a tier fires live, DR

Why this slide is in the deck at all. A number in our own material was wrong by a factor of seven, in the direction that flattered nobody. How a team handles its own bad number is worth more signal than the number was — and the fix is not "measure more carefully next time", it is a script that reads the CloudFormation template and fails CI when these figures drift from what the brief says.

The control that stops it recurring

`tools/observability_census.py` reads the synthesized template and counts what actually deploys. `--assert-baseline` fails if the numbers drift from what the partner documents quote, so the brief and the infrastructure cannot disagree silently. Ask your SA to run it.

THE MOVE

Show them how you behave when your own number is wrong. This slide is about method, not alarms.

SAY

- Say the claim first, then the correction, and do not soften either: we told people three alarms. Twenty-two were deployed. Twenty-six now.
- The error was the METHOD, not the arithmetic: we counted `cw.Alarm()` call sites in the source - two of the three are inside loops - instead of counting what the synthesized template actually deploys.
- Then the control: a script that reads the template and counts what deploys, with an `--assert-baseline` mode that fails if the numbers drift from what the partner documents quote. The brief and the infrastructure cannot disagree silently any more.
- Invite them to run it. It takes seconds and it is the cheapest trust-building step on offer.

DON'T

- Don't frame this as a win. It was wrong in the direction that flattered nobody - we called our strongest-growing area our weakest - and the point is what happened next.

Q&A

Q: Why is a mistake in a partner deck at all?

A: Because you are going to check my numbers, and I would rather you find that I check them first. How a team handles its own bad number is worth more signal than the number was.

Q: What else might be wrong by a factor of seven?

A: The honest answer is: anything that is not gated. That is why the numbers in this deck are the gated ones - the suite size, the alarm count, the account-id redaction, the release tag and the silenced-error scan all fail CI on drift. Where a number is not gated I say so, and the maturity file records which is which.

Q: Is 26 alarms enough?

A: I don't know, and neither does anyone until a tier fires in a real incident. What I can tell you is that they are NAMED - nine of them previously existed only as CloudFormation logical ids, so no runbook could reference them and no operator could find them in the console - they are tiered, and each has a documented first action enforced by a test.

Q: You say this is the same defect as two other register entries. Which?

A: Measuring the generating code instead of the deployed artifact. It shows up as L60 and L67, and it showed up again in the connector work. The standing rule that came out of it is the one on the architecture diagram: measure the artifact, not the source.

THE ASK

One thing, and it is cheap: run it yourself

“All of the evidence is ours, which means none of it counts. We want your SA to run the verification protocol in a clean account. It is scripted, it takes about half a day, and it either reproduces our results or it does not.”

1	Clone the tag	10 min	v0.7.0-pilot-rc1. Confirm tag == main and the RELEASE file inside the tag names the tag.
2	Run the offline suite	20 min	510 pass, 4 skip (514 collected). Confirm the CI security gates are blocking, not report-only.
3	Read a pass and a failure	20 min	A pass and the failure before it, side by side, in the same commit. Cheapest high-signal step in the list.
4	Deploy from zero	~2 hr	Into a clean account you control, following docs/INDEPENDENT-VERIFICATION.md.
5	Run the gate yourself	~1.5 hr	Record the result — including if it fails. That is the deliverable either way.
6	Try to break it	~1 hr	Forge a sanitized_ref. Call a tool with no entitlement. Call finalize_determination directly.
7	Tear down	30 min	Confirm zero stack and zero AgentCore residue in your account.

One prerequisite we owe you: the account needs CloudTrail headroom. The limit is five trails per region and AWS marks it not adjustable. Each deployed pack needs two.

THE MOVE

Make the ask small, concrete, falsifiable - and make clear that a failure is a good outcome.

SAY

- Say the quotation on the slide verbatim. It is the whole meeting in four sentences.
- Then the seven steps, but only emphasise two: step 3 is the cheapest high-signal step, and step 6 is the one I most want them to do.
- Say this plainly: a failure is a USEFUL result. If a step does not work, or you needed a fix that is not in the documentation, that is a real defect and recording it is the entire point.
- And: please do not ask me to talk you through it. That would defeat the exercise.
- Give them the CloudTrail prerequisite BEFORE they start, not after.

DON'T

- Don't offer to run it for them, even to be helpful. See the last question.

Q&A

Q: Half a day of an SA is not nothing. What do we get?

A: A defensible answer to "does this work" that is not mine, and a named list of what stands between it and a customer pilot. If it fails, you have saved us both a customer conversation - that is also a good outcome, and I mean that.

Q: What if our SA finds something embarrassing?

A: Then it goes into the register with their finding, and it gets fixed. That is the seventy-first entry, not a problem. The register exists precisely so that has somewhere to go.

Q: Who pays for the AWS spend?

A: You do, and it is a few dollars - VPC interface endpoints, Lambda, Step Functions, DynamoDB, KMS, one Bedrock call and one Comprehend call. Everything is torn down at the end and your SA confirms zero residue themselves rather than taking our word for it.

Q: Could you just run it for us on a screen share?

A: I could, and it would prove nothing. The entire value is that it runs in an account I do not control, from written instructions, without me in the room. If a step does not work, that is the finding - and if I am on the call, the finding never gets recorded.

Q: What skill level does the SA need?

A: Comfortable with the AWS CLI, CDK bootstrap and Python. The harness is scripted, including a dry-run mode that makes no AWS calls and spends nothing - start there.

HOW THIS MOVES FORWARD

Verify, then extend, then a customer — in that order

A Verify Weeks 1–2	Partner SA reproduces the gate in a clean account, including the break-it step.	<i>Gate: it reproduces, or we learn why not.</i>
B Extend Weeks 3–8	The same connector proof against a real EXTERNAL system of record — ours is proven, a customer's is not. Live re-gate of a second pack. Observability uplift. Organization + SCP if you can supply an OU.	<i>Gate: a system of record we do not own is governed.</i>
C Pilot Quarter 2	Named customer. Supervised pilot on de-identified data. Customer's own security engineering, testing and authorization.	<i>Gate: the customer's AO accepts the compensating-control statement.</i>
D Production Later	Not in scope for this conversation. Pen test, DR, SLO, IdP round-trip, ATO, formal authorization.	—

We are deliberately NOT asking you to find a customer yet. That would be asking you to put your name on synthetic, author-produced evidence.

THE MOVE

Show a sequence with gates, and make the restraint visible: you are not asking them to find a customer.

SAY

- The ORDER is the point: verify, then extend, then a customer. Each phase has a gate that has to be met before the next one starts.
- Phase B is where the connector work goes next: the same proof against a system of record we do NOT own.
- Say the last line out loud, deliberately: we are NOT asking you to find a customer yet. That would be asking you to put your name on synthetic, author-produced evidence.

DON'T

- Don't let phase C slide earlier in the conversation, even if they push. That is the move that ends the relationship badly.

Q&A

Q: Quarter 2 for a pilot feels slow.

A: It is paced by things I do not control: your SA's verification, a real system of record, and a customer's own security engineering. I would rather give you a date I can hit than one that slips. If verification goes well and you can supply an OU, phase B compresses.

Q: What if we want to go straight to a customer?

A: I would push back, and my reason is as self-interested as it is principled: that asks you to put your name on evidence that is synthetic and author-produced. If it goes wrong it goes wrong on your relationship, not mine.

Q: What does "the customer's AO accepts the compensating-control statement" mean?

A: It means the authorizing official agrees the controls present here compensate for what is missing. That is an ATO conversation, not a checkbox, and it needs the customer's own security engineering and testing - which is why it is phase C and not phase A.

Q: What do you need from us in phase B specifically?

A: Three things, in order of value: an OU so the SCP perimeter can be tested for real; an introduction to a system of record we do not own; and an adversarial review of the Cedar policy set and the fault semantics.

Where this actually stands

What it is

A credible governance control plane at reference-architecture maturity, with an unusually well-documented gap list — and further along than most teams will have built themselves.

Good for: architecture workshops, a CISO/AO conversation, a supervised pilot on synthetic or de-identified data.

What it is not

Not a product. Not an AWS service. Not certified, authorized or ATO'd. Not ready for real regulated data. Not independently verified by anyone.

Say this first. It costs nothing, and it buys you the rest of the conversation.

The one thing we need from you

Half a day of an SA's time, in an account we do not control, to find out whether any of this survives contact with someone who is not us.

THE MOVE

Close on the ask, not on the result. Leave them with one decision, not a summary.

SAY

- Say "what it is NOT" first. It costs nothing and it buys you the rest of the conversation - and by this point in the deck it will read as consistency rather than hedging.
- Then what it is: a credible governance control plane at reference-architecture maturity, with an unusually well-documented gap list, and further along than most teams will have built themselves.
- Close on the ask, not on the 21/21: half a day of an SA's time, in an account we do not control, to find out whether any of this survives contact with someone who is not us.
- Then stop talking. The silence is doing work.

DON'T

- Don't re-summarise the proven list. They have seen it, and ending on it undoes the posture the whole deck has built.

Q&A

Q: What would change your mind about any of this?

A: A verification run that does not reproduce. A policy set a customer cannot express in the manifest. Or an auditor who says the evidence chain does not answer their question. Any of those is more useful to me than agreement.

Q: Who else has looked at this?

A: Four external review rounds, and each one found something real - the fault-semantics work in core 1.10.1 came straight out of the third. No partner SA has run it. That is the gap I am asking you to close.

Q: What happens if we say no?

A: I keep building, and the credibility gap stays exactly where it is. I would rather know now than after a customer conversation.

Q: What is the one thing you would want us to remember?

A: That the governance sits in front of the model rather than inside it - and that everything I have claimed today has either a named gate check behind it or a line on the not-proven list. There is no third category.